

# ОБ'ЄКТНО-ОРІЄНТОВАНЕ ПРОГРАМУВАННЯ В КЛІЄНТСЬКИХ ВЕБЗАСТОСУНКАХ

Питання 3.3.

# Принципи SOLID. SRP

---

```
// Account.ts
export class Account {
  private balance: number;

  constructor(balance: number) {
    this.balance = balance;
  }

  public getBalance(): number {
    return this.balance;
  }

  public deposit(amount: number): void {
    this.balance += amount;
  }

  public withdraw(amount: number): void {
    if (amount > this.balance) {
      throw new Error('Insufficient funds');
    }

    this.balance -= amount;
  }
}
```

- SOLID – це аббревіатура від п'яти принципів об'єктно-орієнтованого проектування, метою яких є підвищення зручності супроводу, розширюваності та надійності програмних систем.
- Принцип єдиної відповідальності (SRP) зазначає, що клас повинен мати лише одну причину для змін.
  - Цей принцип допомагає сприяти модульності, зручності супроводу та тестуванню.
  - У TypeScript ви можете використовувати класи та модулі для досягнення SRP.
  - У прикладі клас Account відповідає за управління балансом рахунку, тоді як клас Bank відповідає за управління колекцією рахунків.
  - Розділяючи ці обов'язки на різні класи, ми гарантуємо, що кожен клас має лише одну причину для змін, що робить код більш модульним, зручним у супроводі та тестуванні.

```
// Bank.ts
import { Account } from './Account';

export class Bank {
  private accounts: Account[];

  constructor() {
    this.accounts = [];
  }

  public openAccount(balance: number): Account {
    const account = new Account(balance);
    this.accounts.push(account);
    return account;
  }

  public closeAccount(account: Account): void {
    const index = this.accounts.indexOf(account);
    if (index === -1) {
      throw new Error('Account not found');
    }

    this.accounts.splice(index, 1);
  }
}
```

# Принципи SOLID. SRP на прикладі React

```
// components/UserProfile.tsx
import React, { useState, useEffect } from 'react';

const UserProfile: React.FC = () => {
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetch('/api/user')
      .then(response => response.json())
      .then(data => setUser(data));
  }, []);

  if (!user) return <div>Loading...</div>;

  return (
    <div>
      <h1>{user.name}</h1>
      <p>{user.email}</p>
    </div>
  );
};

export default UserProfile;
```

- У React інтерфейс формується з компонентів – функцій або класів, які повертають опис елементів сторінки.
- Компонент можна розглядати як об'єкт, що:
  - має параметри (props);
  - має внутрішній стан (state);
  - містить методи обробки подій;
  - визначає відображення UI.
- Принципи SOLID застосовуються до структурування поведінки компонентів та взаємодії між ними.
  - У наведеному прикладі визначено функціональний компонент React, який порушує SRP.
  - У початковому варіанті компонент одночасно отримує дані з сервера, обробляє їх та відповідає за відображення.
  - Отже, він має декілька відповідальностей.

# Принципи SOLID. SRP на прикладі React

---

```
// hooks/useUser.ts
import { useState, useEffect } from 'react';

const useUser = () => {
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetch('/api/user')
      .then(response => response.json())
      .then(data => setUser(data));
  }, []);

  return user;
};

export default useUser;
```

- У рефакторинговому варіанті логіка отримання даних винесена в окрему функцію або сервіс, а компонент відповідає лише за відображення.

```
// components/UserProfile.tsx
import React from 'react';
import useUser from '../hooks/useUser';

const UserProfile: React.FC = () => {
  const user = useUser();

  if (!user) return <div>Loading...</div>;

  return (
    <div>
      <h1>{user.name}</h1>
      <p>{user.email}</p>
    </div>
  );
};

export default UserProfile;
```

# Принципи SOLID. ОСР

---

```
// Shape.ts
export interface Shape {
  getArea(): number;
}

// Rectangle.ts
import { Shape } from './Shape';

export class Rectangle implements Shape {
  private width: number;
  private height: number;

  constructor(width: number, height: number) {
    this.width = width;
    this.height = height;
  }

  public getArea(): number {
    return this.width * this.height;
  }
}
```

- ОСР стверджує, що клас має бути відкритим для розширення, але закритим для модифікації.
  - Цей принцип допомагає сприяти розширюваності та зручності підтримки.
  - У TypeScript ви можете використовувати успадкування та інтерфейси для досягнення ОСР.
  - Використовуючи інтерфейс Shape, ми можемо додавати нові фігури до системи без зміни існуючого коду, що робить код більш розширюваним та зручним у підтримці.

```
// Circle.ts
import { Shape } from './Shape';

export class Circle implements Shape {
  private radius: number;

  constructor(radius: number) {
    this.radius = radius;
  }

  public getArea(): number {
    return Math.PI * this.radius ** 2;
  }
}
```

# Принципи SOLID. OCP на прикладі React

---

```
// components/Button.tsx
import React from 'react';

interface ButtonProps {
  onClick: () => void;
  label: string;
}

const Button: React.FC<ButtonProps> = ({ onClick, label }) => {
  return <button onClick={onClick}>{label}</button>;
};

export default Button;
```

- Компоненти в React можуть використовуватись як параметризовані UI-елементи.
  - Вони отримують props – це аналог параметрів конструктора або методу.
- Замість того, щоб щоразу вручну писати HTML-кнопку в різних місцях, розробник створює один універсальний компонент Button і потім використовує його багаторазово.
  - Компонент у React працює не з довільними даними, а з чітко визначеним набором вхідних властивостей.
  - Тут оголошується інтерфейс ButtonProps.
  - React.FC<ButtonProps> – це вказівка, що Button є функціональним компонентом React, який приймає параметри типу ButtonProps.
  - return <button onClick={onClick}>{label}</button>; - це JSX-синтаксис, який використовується в React для опису інтерфейсу.
  - Щоб додати новий тип кнопки, необхідно *змінювати існуючий код*.

# Принципи SOLID. OCP на прикладі React

```
// components/withLogging.tsx
import React from 'react';

const withLogging = <P extends object>(WrappedComponent: React.ComponentType<P>) => {
  return (props: P) => {
    console.log('Rendering', WrappedComponent.name);
    return <WrappedComponent {...props} />;
  };
};

export default withLogging;
```

```
// components/Button.tsx
import React from 'react';
import withLogging from '../withLogging';

interface ButtonProps {
  onClick: () => void;
  label: string;
}

const Button: React.FC<ButtonProps> = ({ onClick, label }) => {
  return <button onClick={onClick}>{label}</button>;
};

export default withLogging(Button);
```

- У withLogging.tsx визначено функцію withLogging, яка приймає готовий компонент і повертає новий компонент із додатковою поведінкою.
  - Перед відображенням компонента в консоль виводиться повідомлення про його рендеринг.
  - Цей код створює механізм розширення вже існуючих компонентів.
  - Функція withLogging() є узагальненою, вона не прив'язана до одного конкретного компонента, наприклад кнопки.
  - Функція отримує інший компонент (WrappedComponent: React.ComponentType<P>), який далі буде розширюватися.
  - Логування додано не через зміну коду кнопки, а через створення окремої обгортки.

# Принципи SOLID. Принцип заміщення Барбари Лісков (LSP)

```
// Vehicle.ts
export class Vehicle {
  public startEngine(): void {
    console.log('Engine started');
  }

  public stopEngine(): void {
    console.log('Engine stopped');
  }
}

// Car.ts
import { Vehicle } from './Vehicle';

export class Car extends Vehicle {
  public drive(): void {
    console.log('Driving');
  }
}

// RaceCar.ts
import { Car } from './Car';

export class RaceCar extends Car {
  public drive(): void {
    console.log('Driving fast');
  }
}
```

- LSP стверджує, що підклас повинен мати можливість заміни батьківського класу без впливу на коректність програми.
  - Іншими словами, ви повинні мати можливість використовувати об'єкт підкласу замість об'єкта батьківського класу без будь-яких проблем. Цей принцип сприяє поліморфізму та розширюваності.
  - У TypeScript ви можете використовувати успадкування та інтерфейси для досягнення LSP.
  - У прикладі клас Vehicle визначає базовий набір методів для транспортного засобу, включаючи запуск та зупинку двигуна.
  - Клас Car розширює клас Vehicle, додаючи метод drive(), специфічний для автомобілів.
  - Клас RaceCar розширює клас Car, додаючи метод drive(), специфічний для гоночних автомобілів.
- Використовуючи успадкування та LSP, ми можемо використовувати об'єкт RaceCar замість об'єкта Vehicle або Car, не впливаючи на коректність програми.
  - Це сприяє поліморфізму та розширюваності, роблячи код більш гнучким та придатним для повторного використання.



# Принципи SOLID. Принцип заміщення Барбари Лісков (LSP) на прикладі React

---

```
// components/Rectangle.tsx
import React from 'react';

interface RectangleProps {
  width: number;
  height: number;
}

const Rectangle: React.FC<RectangleProps> = ({ width, height }) => {
  return <div style={{ width, height, backgroundColor: 'blue' }} />;
};

export default Rectangle;
```



```
// components/Square.tsx
import React from 'react';
import Rectangle from './Rectangle';

interface SquareProps {
  size: number;
}

const Square: React.FC<SquareProps> = ({ size }) => {
  return <Rectangle width={size} height={size} />;
};

export default Square;
```

- Компонент Rectangle відображає прямокутник та очікує два числові параметри – width і height – і на їх основі формує візуальний елемент div із заданими розмірами.
  - Rectangle не містить складної логіки: він просто перетворює два параметри на візуальний результат.
  - Компонент експортується, щоб його можна було використати в інших файлах.
- На відміну від Rectangle, компонент Square приймає лише один параметр – size.
  - Далі цей параметр використовується двічі: і як ширина, і як висота компонента Rectangle.
  - Новий компонент не ламає базовий контракт відображення фігури, а використовує вже наявний компонент відповідно до його правил.
  - Базова поведінка не порушується, новий компонент поводить ся узгоджено з уже наявною моделлю.
  - Відбувається безпечне спеціалізоване використання базового компонента.

# Принципи SOLID. Принцип розділення інтерфейсів (ISP)

```
// Database.ts
export interface Database {
  connect(): void;
  disconnect(): void;
  query(sql: string): any[];
}

// SqliteDatabase.ts
import { Database } from './Database';

export class SqliteDatabase implements Database {
  public connect(): void {
    console.log('Connecting to SQLite database');
  }

  public disconnect(): void {
    console.log('Disconnecting from SQLite database');
  }

  public query(sql: string): any[] {
    console.log(`Executing SQL query: ${sql}`);
    return [];
  }
}
```

- ISP стверджує, що клієнт не повинен бути змушений залежати від методів, які він не використовує.
  - Іншими словами, слід розробляти інтерфейси, специфічні для потреб кожного клієнта, а не створювати монолітний інтерфейс, який включає методи для всіх клієнтів.
  - Цей принцип допомагає сприяти модульності та зручності обслуговування.
  - У TypeScript ви можете використовувати інтерфейси для досягнення ISP.

```
// RedisDatabase.ts
import { Database } from './Database';

export class RedisDatabase implements Database {
  public connect(): void {
    console.log('Connecting to Redis database');
  }

  public disconnect(): void {
    console.log('Disconnecting from Redis database');
  }

  public query(sql: string): any[] {
    throw new Error('Redis does not support SQL queries');
  }
}
```

- У прикладі інтерфейс Database визначає контракт для баз даних, які можуть підключатися, відключатися та виконувати SQL-запити.
- Класи SqliteDatabase та RedisDatabase реалізують інтерфейс Database, надаючи власні реалізації методів.

# Принципи SOLID. Принцип розділення інтерфейсів (ISP) на прикладі React

```
// components/UserCard.tsx
import React from 'react';

interface UserCardProps {
  name: string;
  email: string;
  address: string;
  phoneNumber: string;
}

const UserCard: React.FC<UserCardProps> = ({ name, email, address, phoneNumber }) => {
  return (
    <div>
      <h2>{name}</h2>
      <p>{email}</p>
      <p>{address}</p>
      <p>{phoneNumber}</p>
    </div>
  );
};

export default UserCard;
```

- Компонент UserCard відображає інформацію про користувача, отримує об'єкт параметрів (props), що описується інтерфейсом UserProps.
- Принцип ISP говорить, що клієнт не повинен залежати від інтерфейсів, які він не використовує.
  - У цьому прикладі компонент залежить від phone та address, але не використовує їх.
  - Наслідками стають зайва зв'язність, складніше повторне використання та тестування, більше когнітивне навантаження на розробника.
  - Наприклад, щоб показати тільки email-картку, треба все одно вигадувати телефон і адресу.

# Принципи SOLID. Принцип розділення інтерфейсів (ISP) на прикладі React

```
// components/UserCard.tsx
import React from 'react';

interface UserBasicInfoProps {
  name: string;
  email: string;
}

const UserBasicInfo: React.FC<UserBasicInfoProps> = ({ name, email }) => {
  return (
    <div>
      <h2>{name}</h2>
      <p>{email}</p>
    </div>
  );
};

export default UserBasicInfo;
```

- Тут створено новий компонент UserBasicInfo.
  - Він робить те саме відображення, але використовує менший інтерфейс.
  - Компонент залежить тільки від потрібних даних, має мінімальний контракт, легше комбінується та тестується.
  - У React інтерфейси часто сегрегуються через props.
  - У класичному ООП це виглядало б як:
    - interface UserBasicInfo
    - interface UserFullInfo

# Принципи SOLID. Принцип інверсії залежності (DIP)

---

```
// Logger.ts
export interface Logger {
  log(message: string): void;
}

// ConsoleLogger.ts
import { Logger } from './Logger';

export class ConsoleLogger implements Logger {
  public log(message: string): void {
    console.log(message);
  }
}

// Service.ts
import { Logger } from './Logger';

export class Service {
  private logger: Logger;

  constructor(logger: Logger) {
    this.logger = logger;
  }

  public doSomething(): void {
    this.logger.log('Doing something');
  }
}
```

- У DIP зазначено, що високорівневі модулі не повинні залежати від низькорівневих модулів.
  - Натомість, обидва повинні залежати від абстракцій.
  - Іншими словами, ви повинні розробляти свою систему таким чином, щоб високорівневі модулі можна було легко замінювати іншими модулями, які реалізують ті самі абстракції.
  - Цей принцип допомагає сприяти розширюваності та зручності обслуговування.
  - У TypeScript ви можете використовувати інверсію керування (IoC) та впровадження залежностей (DI) для досягнення DIP.
  - У прикладі інтерфейс `Logger` визначає контракт для логгерів, які можуть реєструвати повідомлення.
  - Клас `ConsoleLogger` реалізує інтерфейс `Logger`, надаючи власну реалізацію методу `log()`.
  - Клас `Service` залежить від інтерфейсу `Logger`, а не від конкретної реалізації логгера.
  - Це дозволяє нам легко замінити `ConsoleLogger` іншою реалізацією логгера, яка реалізує той самий інтерфейс `Logger`.

# Принципи SOLID. Принцип інверсії залежності (DIP) на прикладі React

---

```
// services/UserService.ts
export class UserService {
  async getUser() {
    const response = await fetch('/api/user');
    return response.json();
  }
}
```

- Клас UserService відповідає за отримання даних користувача з API.
  - Він інкапсулює низькорівневу деталь – HTTP-запит через `fetch('/api/user')`.
  - Цей лістинг сам по собі ще не є порушенням чи реалізацією DIP – це просто конкретна реалізація джерела даних.
  - Важливе питання: хто саме буде залежати від цього класу.
  - Якщо UI-компоненти безпосередньо імпортують і викликають UserService, то вони залежатимуть від конкретної реалізації доступу до даних.
  - Саме тому слід перейти до контексту як до більш абстрактного механізму постачання даних.

# Принципи SOLID. Принцип інверсії залежності (DIP) на прикладі React

---

```
// context/UserContext.tsx
import React, { createContext, useContext, useState, useEffect } from 'react';

const UserContext = createContext<any>(null);

export const UserProvider: React.FC = ({ children }) => {
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetch('/api/user')
      .then(response => response.json())
      .then(data => setUser(data));
  }, []);

  return <UserContext.Provider value={user}>{children}</UserContext.Provider>;
};

export const useUser = () => useContext(UserContext);
```

- У цьому файлі створено контекст користувача, призначення якого – один раз отримати дані користувача й далі зробити їх доступними будь-якому вкладеному компоненту без прямого передавання через параметри.
  - Context – це механізм React для спільного доступу до даних між компонентами.
  - Provider – компонент, який “постачає” значення в контекст.
  - Хук useContext(...) – спосіб отримати це значення в будь-якому дочірньому компоненті.
- Замість того, щоб кожен компонент сам ходив у мережу або отримував дані через довгий ланцюг параметрів, React дозволяє створити спільне джерело даних.
  - Це той рівень абстракції, на який починає спиратися інтерфейс.

# Принципи SOLID. Принцип інверсії залежності (DIP) на прикладі React

---

```
// components/UserProfile.tsx
import React from 'react';
import { useUser } from '../context/UserContext';

const UserProfile: React.FC = () => {
  const user = useUser();

  if (!user) return <div>Loading...</div>;

  return (
    <div>
      <h1>{user.name}</h1>
      <p>{user.email}</p>
    </div>
  );
};

export default UserProfile;
```

- Компонент UserProfile відображає дані користувача, але сам більше не відповідає за їх завантаження.
  - Він звертається до абстракції useUser() і працює з уже наданими даними.
  - Саме це є головною ідеєю DIP: компонент залежить не від конкретного HTTP-механізму, а від абстрактного джерела даних.



# Композиція та повторне використання коду

```
export class Monster {
  constructor() {
    if (this.constructor === Monster) {
      throw new Error('This is an abstract class. Do not instantiate it directly.');
```

```
    }
  }

  attack() {
    playHurtAnimation()
  }

  die() {
    userInventory.add(this.dropLoot())
    removeFromUi(this)
  }

  dropLoot() {
    throw new Error('This is an abstract function. It should be shadowed.')
```

```
  }
}

export class Slime extends Monster {
  dropLoot() {
    return [new SlimeBall()]
  }
}

export class Skeleton extends Monster {
  dropLoot() {
    return [new Bone()]
  }
}
```

Перше, про що зазвичай говорять у контексті повторного використання коду, є механізм наслідування.

- Наслідування історично поєднувало не лише повторне використання, але й реалізації поліморфізму, динамічну диспетчеризацію (dynamic dispatch – реалізація методу визначається під час виконання програми, а не під час компіляції), розширення поведінки.
- У коді реалізується ієрархія монстрів у грі.
- Вона інтуїтивно підштовхує до наслідування, бо є спільна поведінка, але окремі типи мають власні особливості.
- Проте класичні проблеми наслідування, на зразок сильної зв'язності та проблеми крихкого базового класу, можуть стимулювати пошук альтернатив.

```
export class Clown extends Monster {
  dropLoot() {
    return [new Key()]
  }

  attack() {
    super.attack()
    if (itsTime()) {
      showCutScene()
    }
  }
}
```

# Композиція та повторне використання коду. Composition over inheritance

```
class MonsterBehaviors {  
  #dropLoot  
  constructor({ dropLoot }) {  
    this.#dropLoot = dropLoot  
  }  
  
  attack() {  
    playHurtAnimation()  
  }  
  
  die() {  
    userInventory.add(this.#dropLoot())  
    removeFromUi(this)  
  }  
}
```

```
export class Skeleton { ... }
```

```
export class Clown { ... }
```

```
export class Slime {  
  #monsterBehaviors  
  constructor() {  
    this.#monsterBehaviors = new MonsterBehaviors({  
      dropLoot: () => this.dropLoot(),  
    })  
  }  
  
  dropLoot() {  
    return [new SlimeBall()]  
  }  
  
  attack() {  
    this.#monsterBehaviors.attack()  
  }  
  
  die() {  
    return this.#monsterBehaviors.die()  
  }  
}
```

- Проста формула “замінімо наслідування композицією” є надто грубою: наслідування об’єднує одразу кілька механізмів, і замінювати доводиться не одну річ, а кілька різних властивостей одразу.
  - Композиція сама по собі не є повною заміною наслідування; вона лише один із інструментів повторного використання.
  - Початковою альтернативою подамо *класичну композицію*: спільну логіку виносять в окремий об’єкт або клас-помічник, а конкретні сутності делегують йому частину роботи.
  - Ідея проста: те, що раніше жило в базовому класі, можна винести в окрему залежність і підключати явно.

# Композиція та повторне використання коду. Composition over inheritance

```
const monsterBehaviors = {
  attack() {
    playHurtAnimation()
  },
  die({ monsterId, dropLoot }) {
    userInventory.add(dropLoot())
    removeFromUi(monsterId)
  },
}
```

```
export class Slime { ... }
```

```
export class Skeleton { ... }
```

```
export class Clown {
  #id = Math.random()

  dropLoot() {
    return [new Key()]
  }

  attack() {
    monsterBehaviors.attack()
    if (itsTime()) {
      showCutScene()
    }
  }

  die() {
    return monsterBehaviors.die({
      monsterId: this.#id,
      dropLoot: () => this.dropLoot(),
    })
  }
}
```

- Для такої спільної логіки не обов'язково створювати окремий клас, якщо жорстко не триматися за тезу “все має бути класом”.
  - Ту саму роль можуть виконувати звичайні допоміжні (helper) функції.
  - Це зменшує обсяг конструкцій і показує, що мета – не дотримання форми, а виділення повторно використовуваної поведінки.
- Практичну проблема: якщо кожен клас мусить заново прокидати делегування до helper-логіки, рішення стає багатослівним і гірше масштабується.
  - Тому частину допоміжної поведінки можна зробити публічною, щоб не дублювати обгортки на кожній сутності; при цьому іншу частину, яку не слід викликати напряму, залишають приватною.

# Композиція та повторне використання коду. Composition over inheritance

---

```
// Public helpers
export const monsterBehaviors = {
  kill(monster) {
    userInventory.add(monster.dropLoot())
    removeFromUi(this)
  }
}

// private helpers
const monsterHelpers = {
  attack() {
    playHurtAnimation()
  }
}
```

```
export class Slime { ... }

export class Skeleton { ... }

export class Clown {
  dropLoot() {
    return [new Key()]
  }

  attack() {
    monsterHelpers.attack()
    if (itsTime()) {
      showCutScene()
    }
  }
}
```

- Приклад відкритих (публічних) та закритих (приватних) допоміжних функцій.
  - Комбінувати різні рішення можна відповідно до ваших потреб.
  - У цьому випадку ми не хотіли публічно розкривати допоміжну функцію `attack()`, оскільки кінцевому користувачеві ніколи не потрібно буде викликати цю функцію безпосередньо.

# Композиція та повторне використання коду. Composition over inheritance

---

```
// Public helpers
export const monsterBehaviors = {
  kill(monster) {
    userInventory.add(monster.dropLoot())
    removeFromUi(this)
  }
}

// private helpers
const monsterHelpers = {
  attack() {
    playHurtAnimation()
  }
}
```

```
export class Slime { ... }

export class Skeleton { ... }

export class Clown {
  dropLoot() {
    return [new Key()]
  }

  attack() {
    monsterHelpers.attack()
    if (itsTime()) {
      showCutScene()
    }
  }
}
```

- Приклад відкритих (публічних) та закритих (приватних) допоміжних функцій.
  - Комбінувати різні рішення можна відповідно до ваших потреб.
  - У цьому випадку ми не хотіли публічно розкривати допоміжну функцію `attack()`, оскільки кінцевому користувачеві ніколи не потрібно буде викликати цю функцію безпосередньо.

# Композиція та повторне використання коду. Composition over inheritance

```
class Monster {
  #onAttack
  constructor({ dropLoot, onAttack, customBehaviors = {} }) {
    this.dropLoot = dropLoot
    this.#onAttack = onAttack ?? (() => {})
    this.behaviors = customBehaviors
  }

  attack() {
    playHurtAnimation()
    this.#onAttack()
  }

  die() {
    userInventory.add(this.dropLoot())
    removeFromUi(this)
  }
}
```

```
const slimeBehavior = { ... }
```

```
const skeletonBehavior = { ... }
```

```
export const createSlime = () => new Monster(slimeBehavior)
export const createSkeleton = () => new Monster(skeletonBehavior)
export const createClown = () => new Monster(clownBehavior)
```

```
const clownBehavior = {
  dropLoot() {
    return [new Key()]
  },
  onAttack() {
    if (itsTime()) {
      showCutScene()
    }
  },
  customBehaviors: {
    uniqueClownBehavior() {
      ...
    }
  },
}
```

- Патерн Стратегія теж може виступати як заміна типового для наслідування заміщення методу в підкласі.
  - Замість неявного очікування override можна зробити явні точки розширення: передавати варіативну поведінку в конструктор або конфігурацію.
  - У результаті кастомізація стає не прихованим ефектом ієрархії, а чітко оголошеним контрактом налаштування.
- У наведеному прикладі ми йдемо до крайності та повністю видаляємо класи slime, skeleton та clown, тому залишається лише один клас Monster, який можна налаштувати.
  - Для демонстрації показуємо, як можна додавати власні методи до певного типу монстра.
  - Будь-хто, хто має екземпляр clown, може викликати clownInstance.behaviors.uniqueClownBehavior().
  - Це не єдиний спосіб використання шаблону стратегії, його також можна комбінувати та поєднувати з іншими методами повторного використання коду.

# Композиція та повторне використання коду. Composition over inheritance

```
const monsterHelpers = {
  attack() {
    playHurtAnimation()
  }
}

class MonsterBehaviors {
  #id = Math.random()
  #dropLoot

  constructor({ dropLoot }) {
    if (this.constructor === MonsterBehaviors) {
      throw new Error('This is an abstract class. Do not instantiate')
    }
    this.#dropLoot = dropLoot
  }

  die() {
    userInventory.add(this.#dropLoot())
    removeFromUi(this.#id)
  }

  // Other shared functions could be defined here as well.
}
```

```
export class Clown extends MonsterBehaviors {
  constructor() {
    super({
      dropLoot: () => this.dropLoot()
    })
  }

  dropLoot() {
    return [new Key()]
  }

  attack() {
    monsterHelpers.attack()
    if (itsTime()) {
      showCutScene()
    }
  }
}
```

```
export class Slime extends MonsterBehaviors { ... }

export class Skeleton extends MonsterBehaviors { ... }
```

- Крихкість базового класу пов'язана з заміщенням методів.
- При ізоляції властивостей та відмові від заміщення базові класи стають ближчими до звичайного набору спільних helper-функцій.
- Надмірна зв'язаність виникає не від самого факту повторного використання, а від зайвого, публічно видимого ієрархічного зв'язку.

- Хоча попередні рішення чудово працюють, вони часто вимагають зміни форми вашого класу, що може не спрацювати для кожного сценарію.
  - Іноді ви просто хочете, щоб усі ваші спільні методи існували окремо на одному об'єкті, як це було б із наслідуванням.
  - Тому для усунення основних недоліків наслідування, його варто **обмежити (restrict)** за правилом PAIL.

## ■ PAIL:

- Private implementation detail – наслідування має залишатися приватною деталлю реалізації;
- Abstract base classes;
- Isolate your base and child classes – базові й дочірні класи слід ізолювати один від одного;
- Inheritance is a Last resort – наслідування має бути “останнім засобом”, коли інші варіанти не підходять.

# Композиція та повторне використання коду. Composition over inheritance

---

- Для поліморфізму не обов'язково використовувати наслідування; достатньо інтерфейсів або контрактів, яким відповідають різні об'єкти.
  - Поліморфізм – це здатність викликати однаковий інтерфейс, а не факт існування базового класу.
  - Наслідування зручне не тому, що воно забезпечує повторне використання, а тому що воно *автоматично дає динамічну диспетчеризацію*.
  - Тобто система може працювати з абстрактним типом, але виконувати конкретну поведінку.
  - Проте динамічну диспетчеризацію (вибір поведінки в рантаймі) можна реалізувати і без наслідування, зокрема через:
    - проєктування на основі інтерфейсів;
    - качину типізацію;
    - ін'єктування стратегій;
    - передачу функцій.
- Вікрита ієрархія – блокувальник архітектури.
  - Коли наслідування стає частиною публічного API, код починає залежати від ієрархії, а не лише від поведінки.
  - Навіть якщо ієрархія перестає бути оптимальною, її складно прибрати. Це інертність дизайну кодової бази.



# Композиція та повторне використання коду. Composition over inheritance

```
const monsterSentinel = {}

export const isMonster = obj => obj.typeSentinel === monsterSentinel

export class Slime {
  typeSentinel = monsterSentinel
  ...
}

export class Skeleton {
  typeSentinel = monsterSentinel
  ...
}

isMonster(new Slime()) // true
isMonster(new Skeleton()) // true
isMonster({ x: 2 }) // false
```

Перевірки типів у рантаймі дають переваги, проте це не аргумент щодо використання наслідування.

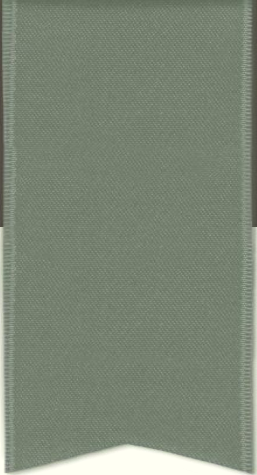
- Райнтайм-перевірки ієрархічного зв'язку замість наслідування можуть реалізовуватись через type sentinels.
- Sentinel – це унікальний об'єкт-маркер, який використовується для визначення належності об'єкта до певної “категорії”.
- Кожен об'єкт-“монстр” має поле:  
`this.typeSentinel = monsterSentinel;`
- Такий «тип» дуже швидко перевіряється та не «підроблюється» випадково, проте потребує дисципліни від розробника, не є типобезпечним та має слабку discoverability,
- З іншого боку, він не потребує ієрархії чи базового класу, є runtime-friendly та може використовуватись у плоских об'єктах (plain objects).
- Альтернативами є:
  - поля-мітки (tags): `this.type = "monster";`
  - контрактний (contract-based) поліморфізм: об'єкт вважається “монстром”, якщо він підтримує потрібний інтерфейс.  
`if (typeof obj.attack === "function") ...`; основа качиної типізації;
  - структурні перевірки:

```
function isMonster(obj) {
  return obj &&
    typeof obj.attack === "function" &&
    typeof obj.die === "function";
}
```

# Композиція та повторне використання коду. Composition over inheritance. Висновки

---

- Наслідування не є лише механізмом повторного використання коду.
  - Воно одночасно забезпечує розширення поведінки, динамічну замінність об'єктів і зручну ідентифікацію типів.
  - Тому композиція не може виступати повною універсальною заміною наслідування.
- Альтернативні підходи дозволяють вирішувати окремі задачі без побудови ієрархій:
  - повторне використання → композиція, helper-функції, делегування;
  - розширення поведінки → strategy та конфігурація об'єктів;
  - перевірка належності до типу → marker (sentinel), tag, helper-перевірки.
- Водночас наслідування залишається корисним інструментом, якщо використовується обмежено:
  - як внутрішня деталь реалізації;
  - без агресивного заміщення поведінки;
  - коли справді існує стабільна ієрархія.
- У сучасних клієнтських системах ефективна організація поведінки досягається не вибором однієї парадигми, а поєднанням композиції, стратегій, допоміжних механізмів та обмеженого наслідування залежно від задачі.



# **ДЯКУЮ ЗА УВАГУ!**

**Наступна тема: Архітектура та модульна організація клієнтського вебзастосунку**